MASTER IASD / DATALAB / ASSIGNMENT 1

What Movie Metadata Adds to Bayesian Matrix Factorization

Collaborative filtering on sparse MovieLens ratings

Group 38 – LHOOQ

DataLab A1 report | October 2026

Group members

Roy Zaworbach

Maryam Shiran

Mahdi Kassem

Course: DataLab **Instructor:** Alexandre Vérine

Université Paris Dauphine–PSL / ENS–PSL

1. The question and how we tested it

CAN MOVIE INFORMATION HELP WHEN MOST RATINGS ARE MISSING?

Our starting point was a familiar problem in recommender systems: a user has rated only a small number of movies, but we would like to predict what they would think of the others. We worked with a user-by-movie matrix containing **610 users and 4,980 movies**. The two provided files each contain 31,598 observed ratings, or about 1% of all possible user–movie pairs. Missing entries are encoded by `NaN`, which we treat as unobserved, not as a rating of zero.

With so few ratings, it is tempting to build a model that is as flexible as possible. But flexibility can also mean overfitting to a handful of opinions. We therefore began with simple baselines and asked whether increasingly structured models could predict unseen ratings more accurately. Our main hypothesis was that **a movie's genre and release period could help estimate its latent factors when its rating history is sparse**.

We compared global/user/movie means, a regularized bias model, user-based KNN, alternating least squares (ALS), plain Bayesian probabilistic matrix factorization (BPMF), and BPMF with side information. We also tried blending KNN with the side-informed model. All models were implemented in NumPy (`models.py`), with dependencies recorded in `requirements.txt`; `generate.py` is the required submission entry point.

Keeping training and evaluation separate

We used two local evaluation protocols for different questions:

1. **Course-style train/test check.** We trained on `ratings_train.npy` and scored only the observed entries in the disjoint `ratings_test.npy`. Our final configuration obtained **RMSE 0.83923** on this test.
2. **Higher-density model comparison.** To approximate the amount of input data provided by the instructor's platform, we merged train and test ratings, then split the observed entries into **64% fitting, 16% calibration, and 20% evaluation** (seed 2026). We fitted blend weights on the calibration part alone, refitted fixed models using fitting plus calibration ratings, and evaluated all methods on the same 12,639 held-out entries. This experiment is in `platform_sim.py`.

We separately ran two 80/20 holdouts (seeds 1 and 2) to compare groups of side features, using `ablation.py`. For any held-out set $\mathcal{T}$, our performance measure is

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i)\in\mathcal{T}} \left(\widehat{r}_{ui} - r_{ui}\right)^2}. \tag{1}$$

The generator clips predicted ratings to $[0.5, 5]$, retains all supplied observed values exactly, and writes a complete matrix to `output.npy`.

Important qualification. The merged-data splits are useful for comparing models under the same conditions, but both released rating files influenced the team's earlier development choices. We therefore treat those results as local development evidence, not as an untouched external test. The hidden platform evaluation remains separate.

2. From simple predictions to latent factors

WHAT WE USED AS A BASELINE, AND WHY WE MOVED BEYOND IT

Biases and nearest neighbours

A reasonable first guess is that some people rate generously and some movies are popular with almost everyone. Our bias baseline uses

$$\widehat{r}_{ui} = \mu + b_u + c_i, \qquad \min_{b,c} \sum_{(u,i)\in\Omega} (r_{ui} - \mu - b_u - c_i)^2 + \lambda_b(\|b\|_2^2 + \|c\|_2^2), \tag{2}$$

where μ is the observed mean and Ω is the set of known entries. The penalty shrinks the estimates for users and movies supported by very few ratings. We used $\lambda_b = 3$ (`fit_biases`).

Our user-based KNN model asks a different question: *when similar users disagree with the baseline, does that tell us something about a missing rating?* We measured user similarity from residuals on co-rated movies, shrank similarities based on overlap, and used the $k = 30$ nearest eligible neighbours for a damped correction to the bias prediction (`user_knn`; damping $= 3$). In this way, a weak or poorly supported neighbour has limited influence.

Alternating least squares (ALS)

Neighbourhoods do not capture every pattern of taste. ALS instead gives each user a vector $p_u \in \mathbb{R}^k$ and each movie a vector $q_i \in \mathbb{R}^k$ and predicts

$$\widehat{r}_{ui} = \mu + b_u + c_i + p_u^\top q_i. \tag{3}$$

We fit these factors by minimizing the squared error *only on observed ratings*:

$$\sum_{(u,i)\in\Omega} (r_{ui} - \mu - b_u - c_i - p_u^\top q_i)^2 + \lambda_b(\|b\|_2^2 + \|c\|_2^2) + \lambda_f \left(\sum_u \|p_u\|_2^2 + \sum_i \|q_i\|_2^2 \right). \tag{4}$$

Holding the movie parameters fixed turns each user update into a small ridge-regression problem, and vice versa. We used 32 latent factors, $\lambda_b = 3$, $\lambda_f = 10$, and 15 alternating passes. Crucially, missing cells never enter the loss as zeros (`als_mf`).

Why we also tried Bayesian matrix factorization

ALS returns one estimated factor vector per user and movie. With sparse data, however, these estimates can be uncertain. BPMF [1] keeps a distribution over the factors and averages predictions from a Gibbs sampler. Writing $U_u = [b_u, p_u]$ and $V_i = [c_i, q_i]$, our model uses

$$r_{ui} \mid U, V, \alpha \sim \mathcal{N}(\mu + b_u + c_i + p_u^\top q_i, \alpha^{-1}), \tag{5}$$

$$U_u \sim \mathcal{N}(m_u, \Lambda_U^{-1}), \qquad V_i \sim \mathcal{N}(n_i, \Lambda_V^{-1}). \tag{6}$$

The factor conditionals are Gaussian; the prior precision matrices and observation-noise precision are updated through Wishart and Gamma conditionals. We used **20 factors, three chains, 20 warm-up sweeps and 80 retained draws per chain**, averaging their rating predictions (`_bpmf_chain`). This increases computation, but it lets us average over sampled factor estimates rather than relying on a single ALS solution.

3. Bringing movie information into BPMF

OUR MAIN EXTENSION: LET PRIOR MEANS DEPEND ON AVAILABLE FEATURES

When a movie has only one or two ratings, its latent vector cannot be estimated reliably from ratings alone. Our idea was to use information about the movie *before* those few observations are taken into account. A comedy and a documentary need not start with exactly the same prior mean, even when both are rarely rated.

Corrected metadata and feature construction

The instructor replaced the original metadata because movie titles and genres were not aligned correctly with the rating columns. We switched to `namesngenre2.npy` and `movie_ids.npy`, checked their expected contents, and used *movie IDs and column positions rather than titles* to identify movies. This matters because some MovieLens titles occur under multiple IDs. The old metadata was **not used** in the results reported here (`movie_metadata`, `verify_metadata.py`).

We built the following features, using only information available to the model at training time:

For	Features
Movies	20 genre indicators, seven release-period indicators parsed from the corrected movie titles, $\log(1 + \text{number of ratings})$, and an intercept.
Users	Average genre/release-period indicators of the movies they have rated, $\log(1 + \text{number of ratings})$, and an intercept.

Counts and user profiles are computed from the visible rating mask, not from held-out ratings. A user's movie-selection profile reflects what they chose to rate; it does *not* tell us directly which genres they enjoyed. If a movie has no parseable year, we use the median year for feature construction.

Changing the prior, not just the final prediction

Standard BPMF pulls user or movie vectors toward a shared population mean. Instead, we let the prior centre vary with available features. For user features g_u and movie features f_i,

$$m_u = g_u^\top W_U, \qquad n_i = f_i^\top W_V, \qquad U_u \sim \mathcal{N}(m_u, \Lambda_U^{-1}), \quad V_i \sim \mathcal{N}(n_i, \Lambda_V^{-1}). \tag{7}$$

Within each sampler sweep, the feature coefficients are estimated by ridge regression,

$$\widehat{W} = (F^\top F + I)^{-1} F^\top X, \tag{8}$$

where F is the feature matrix and X contains the current latent vectors. The fitted feature means are then used in the Gaussian factor updates. Thus, when a movie has few ratings, its prior can reflect its genres and release period; as more ratings arrive, its likelihood has more influence.

There is an important modeling detail here. Our selected configuration uses `prior_mode="plug_in"`: we *estimate* W_U and W_V at each sweep rather than sample them from a joint posterior. The latent vectors and precisions are sampled, but the feature coefficients are not. We therefore describe this as **BPMF with plug-in, feature-informed priors**, not as a fully Bayesian treatment of every parameter. Related ways of incorporating side information exist in the literature [2,3]; our aim in A1 was to evaluate this idea carefully on the provided data, not claim a new recommender architecture.

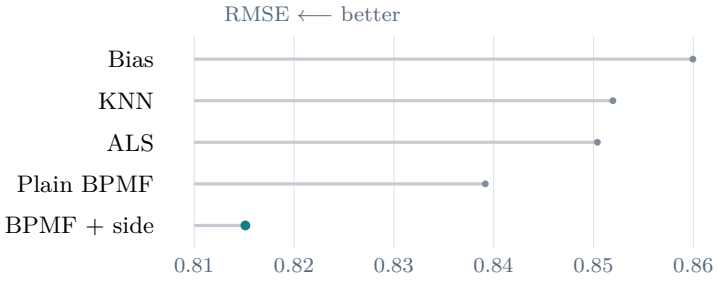
4. What improved, and what did it cost?

MATCHED COMPARISON ON 12,639 HELD-OUT RATINGS

Our main comparison uses the same evaluation cells for every method. After calibration, each base model was fit on the 50,557 visible ratings outside the evaluation split. Table 1 reports the resulting RMSE values and recorded wall-clock prediction times. The hidden platform's scores and runtimes may differ.

Table 1: Local higher-density comparison with corrected metadata, seed 2026. Lower RMSE is better (`results/platform_sim_corrected.csv`).

Method	RMSE	Runtime (s)
Regularized bias baseline	0.85996	0.29
User KNN	0.85194	1.00
ALS (32 factors)	0.85038	2.89
Plain BPMF	0.83915	100.21
BPMF + side information	**0.81513**	**120.01**
Calibrated BPMF–KNN blend	0.81513	—



The progression is encouraging, but the gains are not equal. KNN and ALS improve modestly on the bias baseline. Plain BPMF reduces RMSE further, from 0.85038 (ALS) to 0.83915. The largest additional step comes from the corrected side information: **0.83915 to 0.81513**, a reduction of 0.02402 (about 2.9% relative to plain BPMF). This supports our hypothesis that a better prior is especially useful in a sparse matrix.

The trade-off is speed. In this run, ALS needed under three seconds, while side-informed BPMF took about two minutes. The extra accuracy may be worthwhile when predictions can be computed offline, but the simpler models could be preferable under a tight runtime budget. We tested blending the side-informed BPMF predictor with KNN; the calibration step chose weights of **1.0 for BPMF and 0.0 for KNN**. The final configuration therefore uses BPMF alone.

The course-style train/test result of **0.83923** belongs to a *different* protocol, with fewer training observations. We report it as a separate check, not as a direct comparison with 0.81513.

5. What did the side information contribute?

ABLATION STUDY, LESSONS LEARNED, AND LIMITATIONS

The improvement from side information raised a natural follow-up question: was it mostly the movie metadata, or did rating counts and user profiles matter just as much? We kept the same BPMF settings (20 factors, three chains, 20 warm-up and 80 retained draws) and changed only the available prior features. We repeated the experiment with two random 80/20 splits of the merged observed ratings.

Table 2: RMSE from the corrected-metadata ablation (`ablation.py`, 20 October run). Lower is better.

Feature group(s)	Split 1	Split 2	Mean
None (plain BPMF)	0.8400	0.8411	0.8405
Movie genres + release period	0.8192	0.8183	0.8188
Popularity (rating counts)	0.8373	0.8372	0.8373
User movie-selection profile	0.8386	0.8406	0.8396
All feature groups	**0.8160**	**0.8143**	**0.8152**

What the ablation tells us

Most of the improvement came from the movie descriptors. Genres and release period alone brought mean RMSE down from 0.8405 to 0.8188. Rating-count features and users' movie-selection profiles, when used separately, changed the score much less. One possible explanation is that movie descriptors remain available even when a movie has barely been rated, while a user's selection profile must itself be estimated from sparse behaviour.

Using all the feature groups together improved mean RMSE by another 0.0036 over movie metadata alone, reaching **0.8152**. We saw this pattern in both splits, although with only two holdouts we would not claim a statistically established effect. The ablation also helped us decide where the extra model complexity actually pays off.

What we would be careful about

- **Data reuse.** We designed models using the released train/test data; the merged-data comparisons are not a new external test. The two ablation splits may overlap, so their mean does not provide a confidence interval.
- **Approximate inference.** Our plug-in feature-coefficient updates do not propagate all uncertainty. The number of Gibbs samples is fixed, but we did not carry out a formal mixing or convergence study.
- **Metadata and interpretation.** Title-derived year bins are coarse, and rating a genre is not the same as liking it. The corrected movie-ID alignment was essential; using the old, mismatched file would weaken any conclusion about feature effects.
- **Cost.** BPMF is much slower than ALS or KNN. The hidden platform also reports runtime and exact-rating accuracy, which our local RMSE results do not replace.

Conclusion. On this sparse MovieLens task, the best of the methods we tried was BPMF with feature-informed priors. The ablation points to correctly matched movie genres and release periods as the main source of improvement; user profiles and popularity add a smaller gain. More generally, our experiments suggest that *where we put side information in the model* can matter as much as how many features we include. The final model meets the required matrix-completion interface, but its independent performance will be determined by the instructor's hidden evaluation.

References. [1] R. Salakhutdinov and A. Mnih, "Bayesian Probabilistic Matrix Factorization Using Markov Chain Monte Carlo," *ICML*, 2008. [2] I. Porteous, A. Asuncion and M. Welling, "Bayesian Matrix Factorization with Side Information and Dirichlet Process Mixtures," *AAAI*, 2010. [3] R. P. Adams, G. E. Dahl and I. Murray, "Incorporating Side Information in Probabilistic Matrix Factorization with Gaussian Processes," *UAI*, 2010.